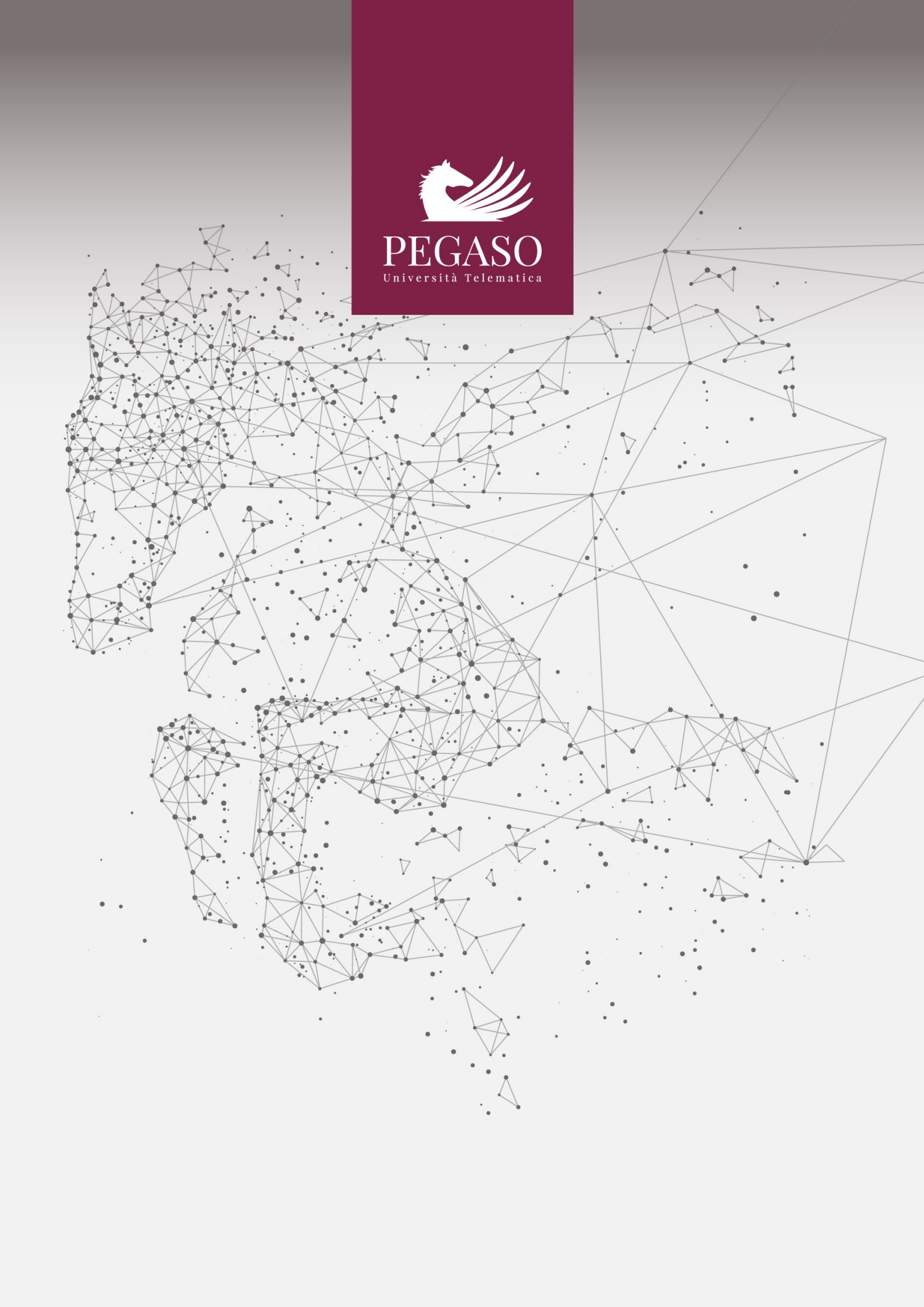

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. LA LIBRERIA MATPLOTLIB 4
3. DECORAZIONI E SALVATAGGIO DEI GRAFICI 14
4. GRAFICI E NUMPY 18
5. SEABORN 22
BIBLIOGRAFIA 24

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano gli strumenti di Python per la rappresentazione grafica dei dati al fine di migliorare la lettura e la comprensione di dati, oltre che per iniziare in modo corretto un'analisi. In particolare, viene introdotta la libreria Matplotlib ed alcune delle tipologie di grafici che essa può calcolare e mostrare (grafico lineare, grafico a torta, grafico a barre, grafico scatter, istogramma). Si presentano, inoltre, le funzionalità per la decorazione, il ridimensionamento e il salvataggio dei grafici su file. Si illustra, poi, il raccordo tra la libreria NumPy e la libreria Matplotlib che permette di graficare gli array generabili anche secondo una funzione matematica. Infine, viene illustrato brevemente Seaborn, la libreria che migliora l'aspetto grafico di Matplotlib ed offre funzioni per grafici avanzati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. La libreria Matplotlib

La **rappresentazione grafica dei dati** è una componente essenziale di qualsiasi attività esplorativa o predittiva di analisi dei dati. Essa è anche la parte più importante delle attività di realizzazione dei report finali. La rappresentazione grafica permette, infatti, di migliorare la lettura e la comprensione di dati, oltre che di iniziare in modo corretto un'analisi.

Matplotlib è la più famosa, importante e completa libreria di grafici per Python. Anaconda è già dotata di tale libreria, quindi non sarà necessario installarla.

Procediamo scrivendo il codice del primo grafico in Python. Per la corretta rappresentazione di un grafico in un notebook Jupyter è necessario scrivere la seguente direttiva:

%matplotlib inline

Inoltre, dovremo importare il modulo pyplot della libreria matplotlib fornendo per brevità d'uso un alias nel seguente modo:

import matplot.pyplot as matplot

Per creare il primo grafico, si utilizzerà la funzione **plot**().Alla funzione plot() verranno passate due liste:

1. la prima per i valori delle **ascisse** (l'asse orizzontale X);
2. la seconda per i valori delle **ordinate** (l'asse verticale Y).

Ovviamente le due liste devono avere la stessa dimensione. Il risultato è un **grafico lineare** che unisce i punti costituiti dall'associazione dei valori collocati in posizioni corrispondenti come indicato nella figura seguente.

PEGASO
Università Telematica

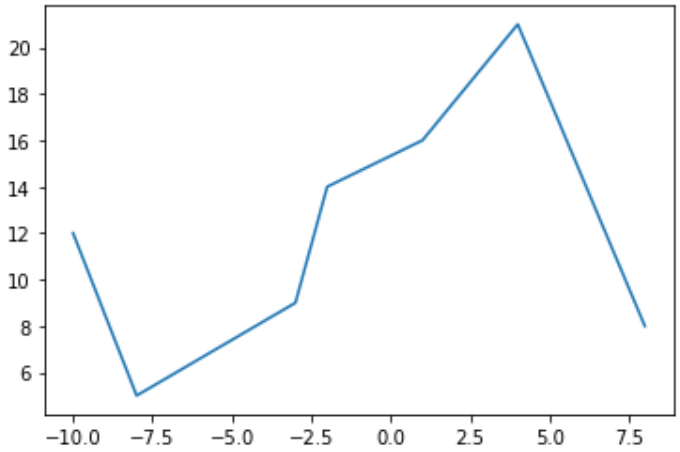
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Prima lista	Seconda lista	Coordinate
-10	12	(-10,12)
-8	5	(-8,5)
-3	9	(-3,9)
-2	14	(-2,14)
1	16	(1,16)
4	21	(4,21)
8	8	(8,8)

Il codice, molto compatto, per calcolare e mostrare il grafico è il seguente:

```
%matplotlib inline
import matplotlib.pyplot as matplot

matplot.plot([-10, -8, -3, -2, 1, 4, 8],[12, 5, 9, 14, 16, 21, 8])
matplot.show()
```



Aumentando gli argomenti è possibile anche **presentare più curve sulla stessa area** sfruttando gli effetti di intersezione. Nell'esempio seguente, plot() riceve sei argomenti che raccolti a coppie rappresentano ascisse e ordinate di tre curve diverse. In particolare, rappresenteremo una parabola, una retta crescente ed una retta decrescente.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per generare la lista dei valori delle ordinate utilizzeremo le **list comprehensions** che semplificano la creazione di liste e sottoliste utilizzando un codice compatto. Tutte e tre le curve avranno valori di ascisse nell'intervallo range(-100,100), mentre:

- la **prima curva** userà i valori delle ascisse per creare una **parabola**. Ogni elemento del risultato di range viene elevato al quadrato attraverso la seguente istruzione:

  *[x**2 for x in range(-100,100)]*

- la **seconda curva** genererà una **retta decrescente** (coefficiente m=-40) attraverso la seguente istruzione:

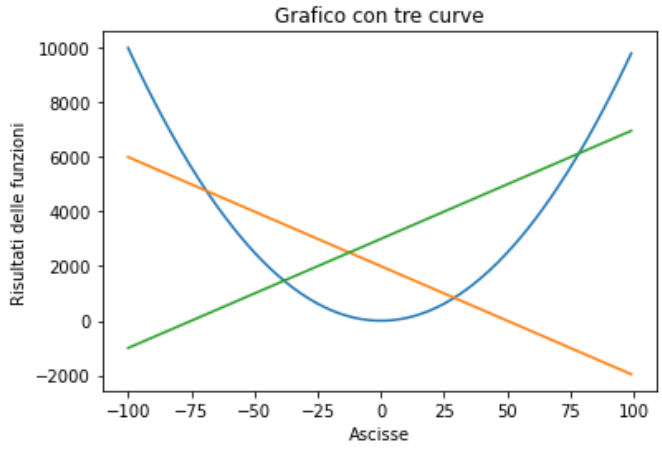
  *[-40*x+2000 for x in range(-100,100)]*

- la **terza curva** genererà una **retta crescente** (coefficiente m=40) attraverso la seguente istruzione:

  *[40*x+3000 for x in range(-100,100)]*

```
%matplotlib inline
import matplotlib.pyplot as matplot

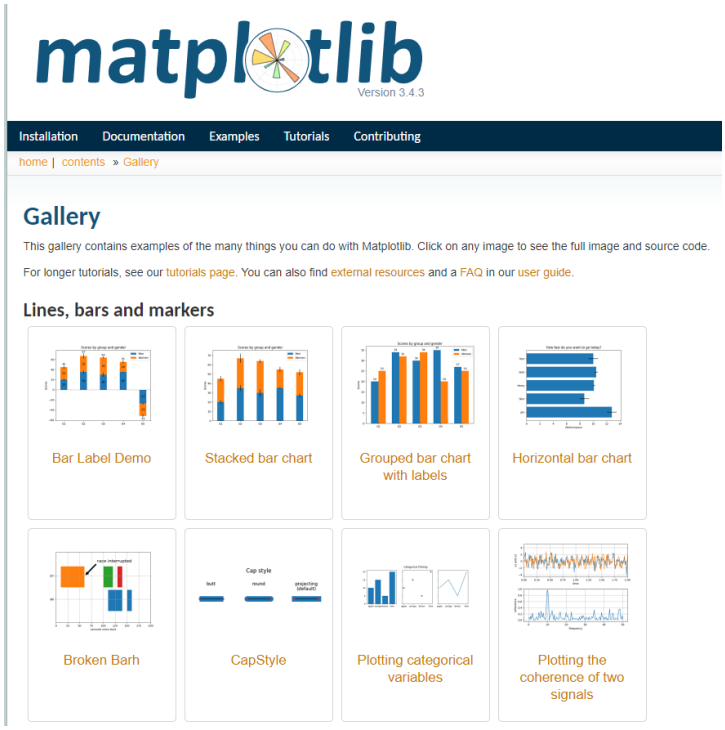
matplot.plot(range(-100,100), [x**2 for x in range(-100,100)],
             range(-100,100), [-40*x+2000 for x in range(-100,100)],
             range(-100,100), [40*x+3000 for x in range(-100,100)],)
matplot.title("Grafico con tre curve")
matplot.xlabel("Ascisse")
matplot.ylabel("Risultati delle funzioni")
matplot.show()
```



PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ovviamente non esistono solo grafici lineari. A titolo conoscitivo, le varie **tipologie di grafico** rese disponibili da Matplotlib sono riportate sul sito ufficiale della libreria disponibile al link: https://matplotlib.org/stable/gallery/index.html#



Cliccando sui box relativi ai grafici, si apre la pagina relativa al grafico selezionato in cui si spiega come generare il grafico e un esempio di codice come mostrato nella figura seguente.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Grouped bar chart with labels

This example shows a how to create a grouped bar chart and how to annotate bars with labels.

```
import matplotlib.pyplot as plt
import numpy as np


labels = ['G1', 'G2', 'G3', 'G4', 'G5']
men_means = [20, 34, 30, 35, 27]
women_means = [25, 32, 34, 20, 25]

x = np.arange(len(labels))  # the label locations
width = 0.35  # the width of the bars

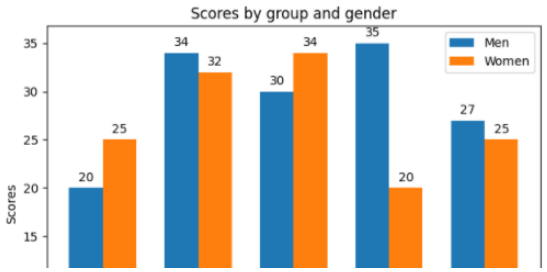
fig, ax = plt.subplots()
rects1 = ax.bar(x - width/2, men_means, width, label='Men')
rects2 = ax.bar(x + width/2, women_means, width, label='Women')

# Add some text for labels, title and custom x-axis tick labels, etc.
ax.set_ylabel('Scores')
ax.set_title('Scores by group and gender')
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()

ax.bar_label(rects1, padding=3)
ax.bar_label(rects2, padding=3)

fig.tight_layout()

plt.show()
```



Analizziamo ora alcuni tipi di grafico. Il **grafico a torta** rappresenta percentuali espresse come fette di un disco.

Per migliorare la visualizzazione si possono aggiungere elementi come ad esempio:

- colorazione;
- ombreggiature;
- distanza tra le fette.

Procediamo, ora, con un esempio di rappresentazione con grafico a torta. Per un sondaggio, si chiede alle persone come amano impiegare le domeniche d'inverno. I risultati sono stati i seguenti:

- Faccio compere (12%);
- Riposo (30%);
- Guardo TV (38%);
- Faccio visita ai parenti (12%);
- Non risponde (8%).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ovviamente, la somma delle percentuali delle risposte deve essere pari al 100%.

Impostiamo per prima cosa le liste delle risposte e dei valori corrispondenti alle risposte stesse. Poi definiamo i seguenti aspetti:

- **explode** è una tupla che determina quanto ogni spicchio della torta deve allontanarsi dagli altri;
- **shadow** = True indica la presenza di ombreggiatura;
- **startangle** è l'entità della rotazione del disco (torta).

```
%matplotlib inline
import matplotlib.pyplot as matplot

risposte = ['Faccio compere',
            'Riposo', 'Guardo TV',
            'Faccio visita ai parenti',
            'Non risponde']
valori = [12, 30, 38, 12, 8]
explode = (0, 0.1, 0.1, 0, 0)

matplot.pie(valori, explode, risposte, shadow = True, startangle = 50)
```

Analizziamo, ora, nel dettaglio la legenda del grafico. Le etichette del grafico vengono generate con la tecnica delle *list comprehensions* unita al *formatted string literals*. In particolare, la tecnica del **formatted string literals** prevede l'utilizzo della *f* prima del letterale di stringa che non sarà costante, ma sarà valutato a run-time in base alla seconda parte dell'istruzione. La parte variabile del letterale di stringa viene inserita tra parentesi graffe. Per ciascuna coppia (risposta, valore) creata attraverso la funzione **zip**(), la **risposta** verrà sostituita ad {a} e il **valore** a {b} che sarà espresso in percentuale tra parentesi tonde. Il **titolo** in legend() è relativo alla legenda e non all'intero grafico per cui il titolo è stato già definito da matplot.title().

La localizzazione della legenda nel grafico si decide attraverso l'argomento **loc** in grado di ricevere stringhe che indicano dove collocare la legenda (es. "upper right", "upper left", "lower right", ecc.). Il parametro **bbox_to_anchor** serve per posizionare la legenda in abbinamento al valore di loc; le coordinate di bbox sono interpretate rispetto all'area del grafico. È possibile, infatti, anche sovrapporre legenda e grafico.

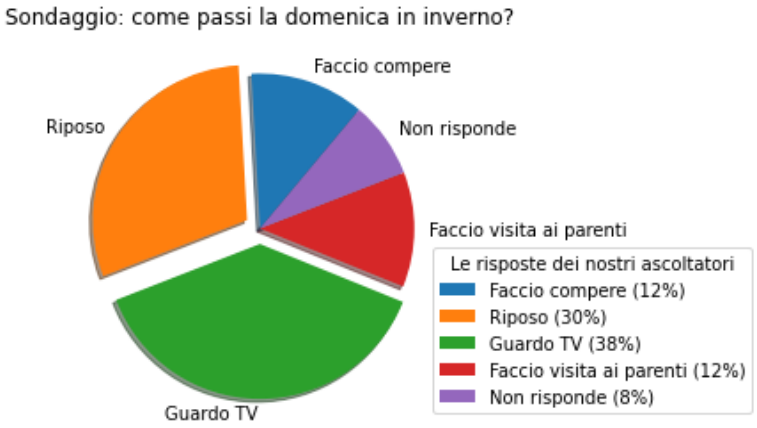
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
matplot.title("Sondaggio: come passi la domenica in inverno?")
matplot.legend([f'{a} ({b}%)' for a,b in zip(risposte, valori)],
               title = "Le risposte dei nostri ascoltatori",
               bbox_to_anchor=(1.8,0),
               loc='lower right')

matplot.show()
```

Il risultato del codice riportato sarà il seguente grafico a torta.



I risultati del sondaggio riportati nel grafico a torta possono essere visualizzati anche mediante un **grafico a barre.** Il metodo da usare in questo caso sarà **bar**() che richiede valori ed etichette in input. Si noto che è possibile passare anche un elenco di colori per la colorazione delle barre del grafico.

PEGASO
Università Telematica

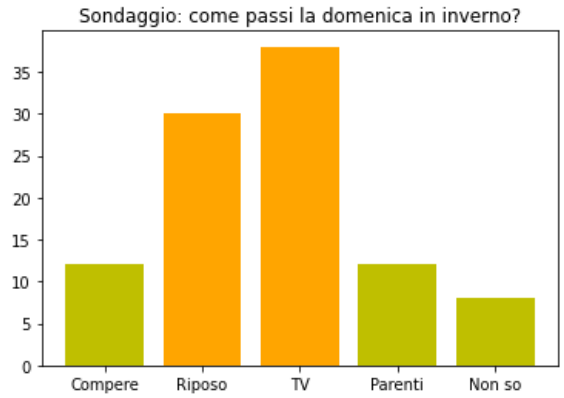
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
%matplotlib inline
import matplotlib.pyplot as matplot

risposte = ['Compere',
            'Riposo', 'TV',
            'Parenti',
            'Non so']
valori = [12, 30, 38, 12, 8]

matplot.bar(risposte, valori, color = ['y', 'orange', 'orange', 'y', 'y'])
matplot.title("Sondaggio: come passi la domenica in inverno?")

matplot.show()
```

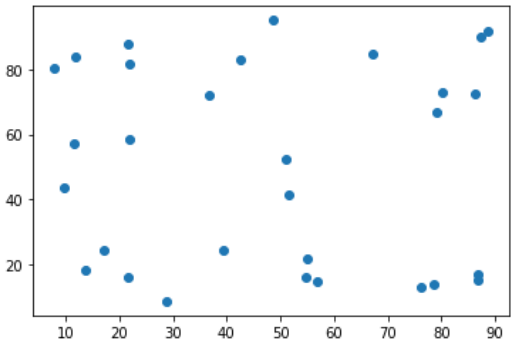


Un altro esempio di grafico è il grafico **scatter** specializzato nel mostrare punti in un piano (funzione **scatter**()). Nell'esempio che segue si visualizzano 30 punti di coordinate casuali nel range [0,100].

```
%matplotlib inline
import matplotlib.pyplot as matplot
import random as rnd

matplot.scatter([rnd.random()*100 for x in range(30)],[rnd.random()*100 for x in range(30)])


matplot.show()
```



Un altro esempio di grafico è l'**istogramma** che non mostra i valori in quanto tali, ma la loro distribuzione nella sequenza di dati e permette di riconoscere velocemente le frequenze dei valori.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

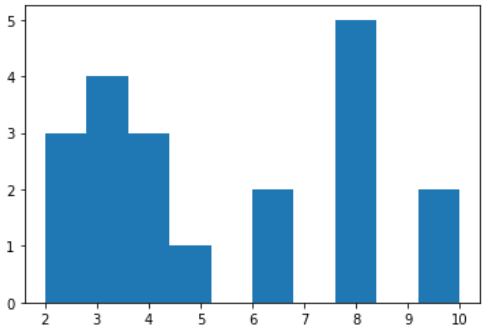
Nel codice che segue, si riporta l'istogramma della distribuzione di 20 numeri interi casuali nell'intervallo 1,9.

```
%matplotlib inline
import matplotlib.pyplot as matplot
import random as rnd

valori = [rnd.randint(1,10) for x in range(20)]
print(valori)
matplot.hist(valori)

matplot.show()
```

```
[4, 8, 8, 2, 8, 6, 3, 8, 10, 2, 2, 4, 10, 5, 3, 4, 3, 3, 8, 6]
```



Altri tipi di grafici di pyplot sono riportati nella tabella seguente insieme alle funzioni che li generano (l'elenco completo è disponibile nella Gallery di matplotlib).

Tipo di grafico	Funzione
Grafico a barre orizzontali	barh()
Grafico a scatola e baffi	boxplot()
Grafico a barre di errore	errorbar()
Grafico log-log	loglog()
Grafico log in X	semilogx()
Grafico log in Y	semilogy()
Grafico per date	plot_dates()
Grafico polare	polar()
Grafico a passi	step()

Matplotlib mette, inoltre, a disposizione la composizione di grafici attraverso la **funzione subplot**() e **subplots**(). Tale funzione suddivide lo spazio del grafico come una matrice e riempie

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ciascuna "cella" con un "sotto-grafico". Nell'esempio seguente si utilizzerà il seguente schema di posizionamento (indici) dei subplot.

0,0	0,1
1,0	1,1

In particolare, nel codice seguente si vogliono riportare i grafici per le quattro funzioni x, x^2, x^3 e x^4 (utilizzando colori diversi) in un unico plot suddiviso in quattro quadranti.

```
import matplotlib.pyplot as matplot
import numpy as np

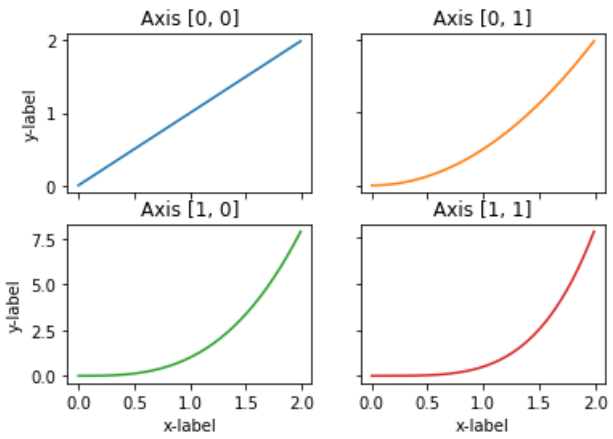
x = np.arange(0.0, 2.0, 0.01)

fig, grafico = matplot.subplots(2, 2)
grafico[0, 0].plot(x, x)
grafico[0, 0].set_title('Axis [0, 0]')
grafico[0, 1].plot(x, x**2, 'tab:orange')
grafico[0, 1].set_title('Axis [0, 1]')
grafico[1, 0].plot(x, x**3, 'tab:green')
grafico[1, 0].set_title('Axis [1, 0]')
grafico[1, 1].plot(x, x**4, 'tab:red')
grafico[1, 1].set_title('Axis [1, 1]')

for ax in grafico.flat:
    ax.set(xlabel='x-label', ylabel='y-label')

#nascondere le x-label e le etichette in alto per i subplot in alto
#e le y-label per i subplot di destra
for ax in grafico.flat:
    ax.label_outer()
```

Il risultato dell'elaborazione del codice precedente sarà:



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Decorazioni e salvataggio dei grafici

Ogni grafico ha un insieme di componenti comuni che possono essere modificati.

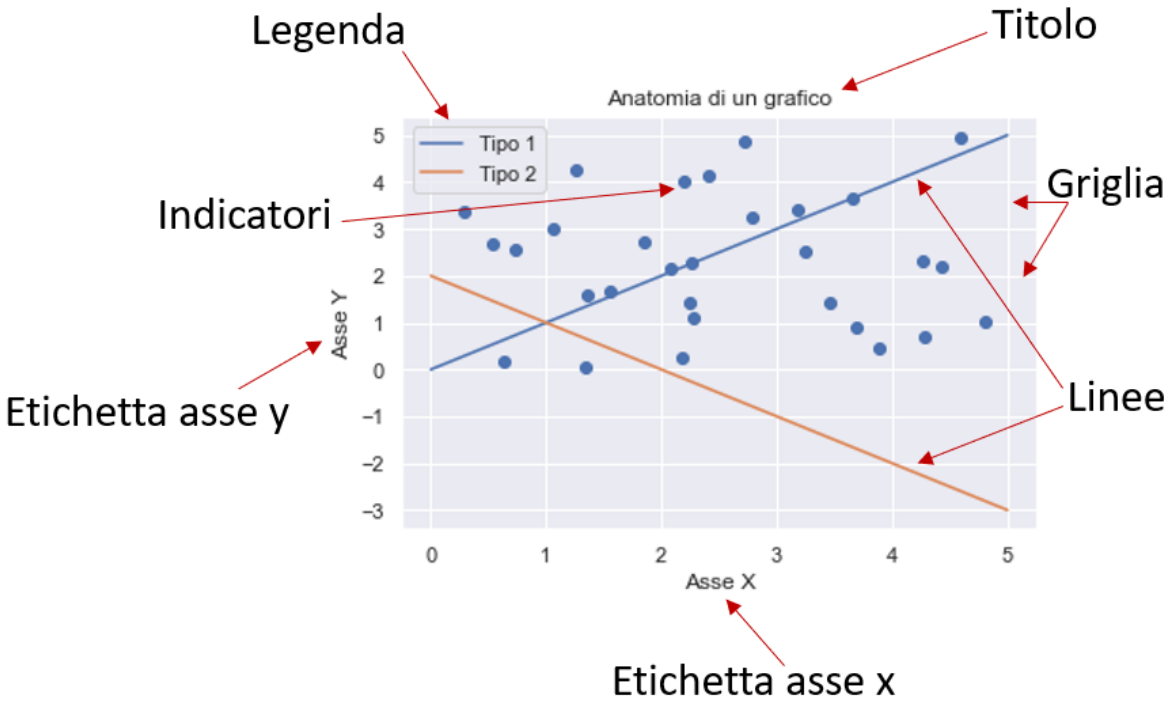
I componenti principali di un grafico sono:

- **Figura** – la base del grafico, su di essa vengono disegnati tutti gli altri componenti;
- **Asse** – contiene gli elementi della figura e imposta il sistema di coordinate;
- **Titolo** – il nome del grafico;
- **Etichetta dell'asse x e dell'asse y** - il nome dell'asse x e il nome dell'asse y, rispettivamente;
- **Legenda** – una descrizione dei dati disegnati nel grafico che permette d'identificare le curve e i punti del grafico;
- **Segni ed etichette dei segni** – i punti di riferimento su una scala; indicano i valori di riferimento;
- **Linee del grafico** – le linee tracciate con i dati;
- **Indicatori** – pittogrammi che contrassegnano i dati;
- **Cornici** – linee che delimitano l'area del grafico in cui vengono tracciati i dati.

Nella figura seguente si riporta l'indicazione di alcuni dei componenti su un grafico di esempio.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Con pyplot è possibile anche controllare diversi aspetti di un grafico. Ad esempio è possibile:

- impostare e modificare la scala degli assi (linear o log, logaritmica) grazie alle funzioni **xscale**(scala) e **yscale**(scala);
- impostare e modificare i limiti degli assi con le funzioni **xlim**(xmin, xmax) e **ylim**(ymin, ymax);
- aggiungere delle note con **annotate**(), delle frecce con **arrow**() e la legenda con **legend**();
- impostare la griglia con **grid**().

Per l'elenco completo delle funzioni di decorazione e dei loro argomenti è possibile far riferimento alla documentazione di pyplot.

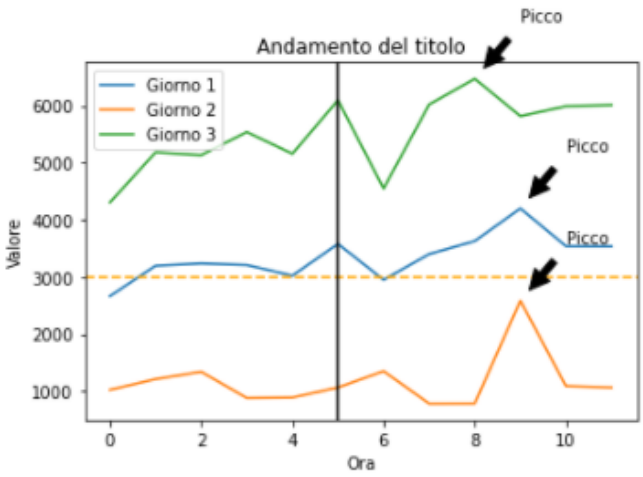
Per visualizzare gli effetti di alcune decorazioni del grafico, si riporta il seguente codice di esempio che prevede l'illustrazione di un grafico di tre curve con l'indicazione della legenda (legend()), del titolo (title()), delle etichette delle ascisse (xlabel()) e delle ordinate (ylabel()), delle frecce e dei testi delle frecce (annotate()), delle linee di riferimento orizzontali (axhline()) e verticali (axvline()).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as matplot
import matplotlib

dati1 = np.genfromtxt('dati_finanziari.txt')
x = range(len(dati1))
rumore = 500 * np.random.normal(size = len(dati1))
dati2 = dati1-2000-rumore
dati3 = dati1+2000+rumore
d = [dati1, dati2, dati3]
giorni = ["Giorno 1", "Giorno 2", "Giorno 3"]
matplot.plot(x,dati1,x,dati2,x,dati3)
for dati in d:
    matplot.annotate(text="Picco", xy=(dati.argmax(), dati.max()),
                     xytext =(dati.argmax()+1, dati.max()+1000),
                     arrowprops= {"facecolor":"black", "shrink":0.2})
matplot.legend(giorni)
matplot.title("Andamento del titolo")
matplot.xlabel("Ora")
matplot.ylabel("Valore")
matplot.axhline(y=3000, color = 'orange', linestyle ='--')
matplot.axvline(x=5, color = 'black', linestyle ='-')
matplot.show()
```



Due funzionalità di estrema importanza messe a disposizione da matplotlib sono:

1. il salvataggio su file dei grafici;
2. la modifica delle dimensioni dei grafici.

Ad ogni modo, queste due funzionalità sono esempi di cosa si può fare accedendo alla **Figure** del grafico. Ogni volta che si dovranno applicare delle configurazioni, infatti, si farà riferimento alla Figure. Esiste il metodo **gcf**() che estrae la Figure corrente o la crea se non esiste ancora. Procediamo, quindi con un esempio di ridimensionamento del grafico. Essendo le misure

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

di default in inches (pollici), è necessario dichiarare *cm* come 1/2.54 e moltiplicarlo per il numero effettivo di centimetri per ottenere la dimensione del grafico.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as matplot

matplot.plot(range(6),[x for x in range(6)])
matplot.show()
fig = matplot.gcf()
cm = 1/2.54
fig.set_size_inches(10*cm,12*cm)
```

Nel codice seguente, si riporta, invece, un esempio di salvataggio del grafico in un file attraverso la funzione **savefig**() che prende come parametri il nome del file e il formato grafico.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as matplot

matplot.plot(range(6),[x for x in range(6)])
#matplot.show()
fig = matplot.gcf()
cm = 1/2.54
fig.set_size_inches(10*cm,12*cm)
matplot.savefig('grafico_retta.jpg', format='jpg')
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Grafici e NumPy

Finora sono state utilizzate normali strutture dati Python come sorgenti di dati. Si presentano ora le potenzialità di Python nel **graficare array NumPy** come fonti di dati.

La libreria Matplotlib è compatibile con Numpy, infatti, basterà passare un array NumPy come argomento di plot per visualizzare un grafico come mostrato nel codice sorgente che segue. Nel codice vengono inserite anche delle istruzioni per far apparire linee orizzontali e verticali nel grafico per migliorarne la comprensione.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as matplot

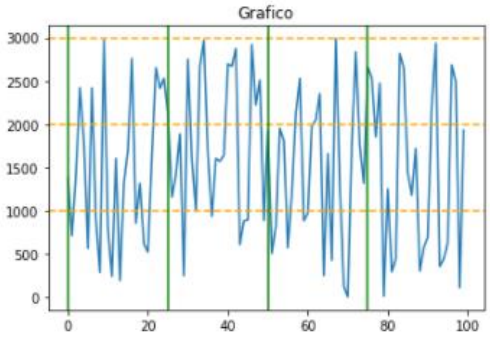
dati = np.random.rand(100)*3000

matplot.title("Grafico")
matplot.plot(dati)

#inseriamo delle linee orizzontali per migliorare la comprensione del grafico
matplot.axhline(y=3000, color = 'orange', linestyle='--')
matplot.axhline(y=2000, color = 'orange', linestyle='--')
matplot.axhline(y=1000, color = 'orange', linestyle='--')

matplot.axvline(x=0, color = 'green', linestyle='-')
matplot.axvline(x=25, color = 'green', linestyle='-')
matplot.axvline(x=50, color = 'green', linestyle='-')
matplot.axvline(x=75, color = 'green', linestyle='-')

matplot.show()
```



Molto interessante è anche la generazione e la successiva visualizzazione di **valori generati secondo una funzione matematica.** In questo esempio genereremo **un'onda sinusoidale di sintesi.** Grazie a NumPy svolgeremo le seguenti operazioni vettoriali:

1. creazione di un array di numeri interi sequenziali;
2. conversione di tali numeri in numeri in virgola mobile;

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. regolazione del periodo;
4. calcolo del seno;
5. amplificazione dei valori;
6. calcolo dello scostamento;
7. aggiunta del rumore gaussiano;
8. troncamento del risultato come se il segnale fosse stato misurato da uno strumento di range limitato.

Eseguiamo le prime sei operazioni con il codice seguente. Definiamo, attraverso delle costanti numeriche:

- l'ampiezza del segnale;
- l'entità dello scostamento;
- il periodo del segnale;
- l'ampiezza del rumore;
- il numero dei campioni totali;
- il range (limitato) dello strumento.

Generiamo l'array del tempo come intervallo di numeri interi della dimensione di 5 volte il periodo del segnale. Successivamente generiamo il segnale come funzione seno del tempo moltiplicata per l'ampiezza del segnale e sommata allo scostamento.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as matplot
import matplotlib

SIG_AMPLITUDE = 10 #ampiezza del segnale
SIG_OFFSET = 2     #entità dello scostamento
SIG_PERIOD = 100   #periodo del segnale
NOISE_AMPLITUDE = 3 #ampiezza del rumore
N_samples = 5 * SIG_PERIOD #numero dei campioni totali
INSTRUMENT_RANGE = 9  #range dello strumento (limitato)

#costruzione dell'onda sinusoidale e aggiunta del rumore
times = np.arange(N_samples).astype(float)
signal = SIG_AMPLITUDE * np.sin(2 * np.pi * times/SIG_PERIOD) + SIG_OFFSET

matplotlib.style.use("ggplot")
matplot.plot(times, signal)
matplot.title("Onda sinusoidale di sintesi")
matplot.xlabel("Tempo")
matplot.ylabel("Segnale")
matplot.ylim(ymin = -SIG_AMPLITUDE-1, ymax = SIG_AMPLITUDE+3)

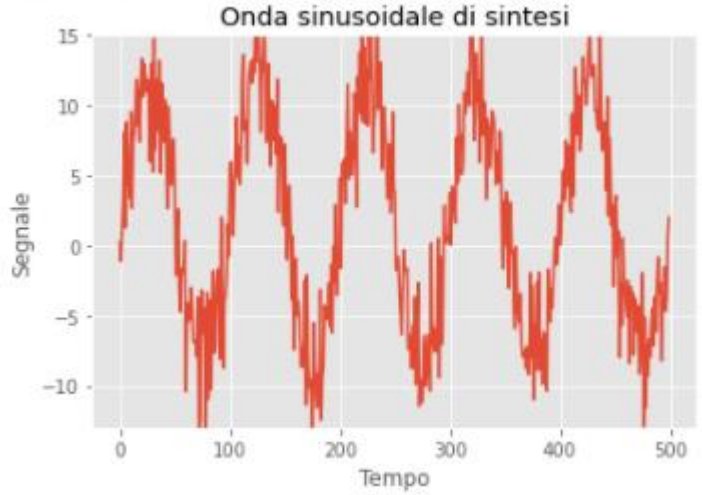
matplot.show()
```



Aggiungiamo del rumore gaussiano al segnale nel seguente modo, ottenendo il grafico.

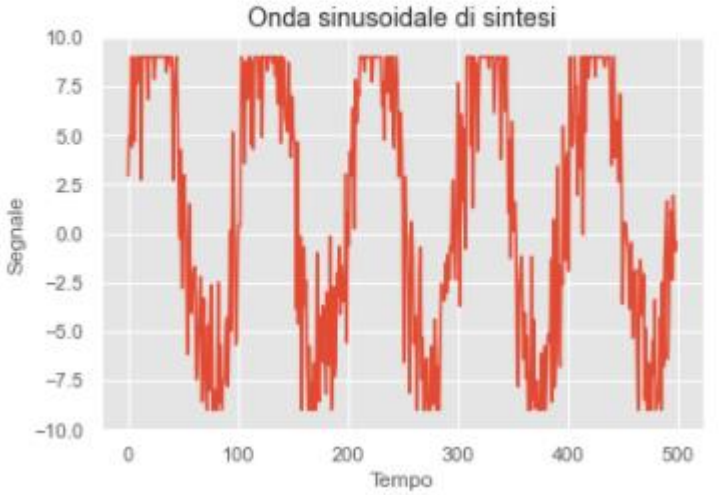
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
noise = NOISE_AMPLITUDE * np.random.normal(size = N_samples)
signal += noise
```



Tronchiamo il risultato come se fosse stato misurato da uno strumento di range limitato (saturazione). Il risultato finale sarà il grafico seguente.

```
#simulazione del range limitato dello strumento
signal[signal > INSTRUMENT_RANGE] = INSTRUMENT_RANGE
signal[signal < -INSTRUMENT_RANGE] = -INSTRUMENT_RANG
```



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Seaborn

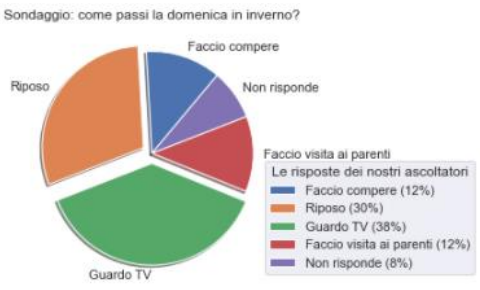
In generale, abbiamo visto che la libreria matplotlib è ricchissima a livello di funzionalità offerte, ma il suo aspetto grafico non è molto moderno. Senza rinunciare alla piattaforma matplotlib è possibile rendere il suo approccio grafico più moderno attraverso un motore grafico che prende il nome di **seaborn** già incluso in Anaconda. Per poterne trarre vantaggio è sufficiente importare il package e attivarlo con il metodo set() che non necessita di parametri.

Di seguito verranno mostrati alcuni grafici precedentemente presentati con la miglioria portata da seaborn.

Esercizio del grafico a torta

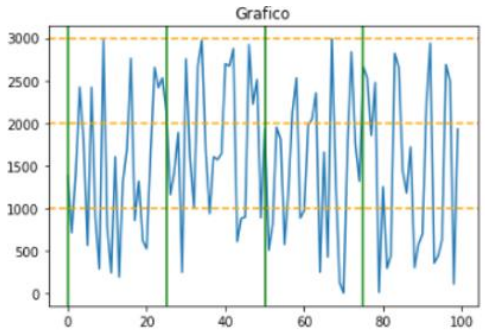


Matplotlib

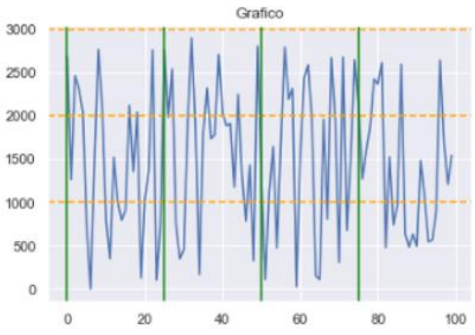


Seaborn

Esercizio dei numeri casuali con NumPy



Matplotlib



Seaborn

Il modulo seaborn permette, inoltre, di mostrare l'istogramma dei valori con la funzione **displot**() e, grazie alla proprietà **kde**, quando uguale a True, di calcolare una stima della densità

PEGASO
Università Telematica

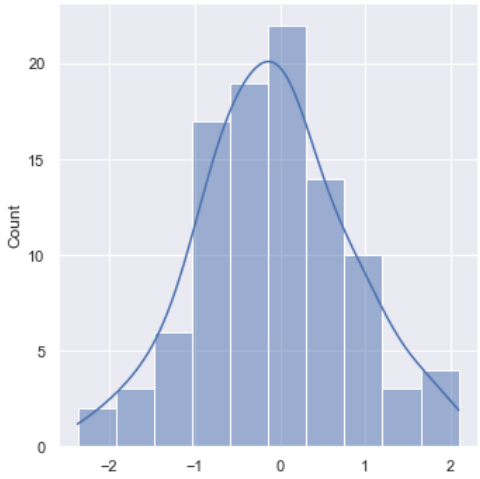
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

del kernel per uniformare la distribuzione e mostrarla sul grafico come osservabile nel codice e nel grafico seguente.

```
%matplotlib inline
import matplotlib.pyplot as matplot
import numpy as np
import seaborn as sb

sb.set()
x = np.random.randn(100)
sb.displot(data = x, kde=True)

matplot.show()
```



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev D. (2017). Data science con Python. Apogeo.
- Maggi G. (2020). Data science con Python. Edizioni LSWR.
- Marin I., Shukla A., Sarang VK. (2019). L'analisi dei Big Data con Python. Tecniche nuove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).